MaLO: Multi-agent LLM Operations for Autonomous Security

Sebastián R. Castro, Iakov Taranenko, Astra Tsai, Nancy Lau, and Alvaro A. Cardenas
University of California, Santa Cruz

***Abstract*—Despite the wide adoption of Large Language Models (LLMs) and AI agents in cybersecurity, the impact of agent topology on autonomous cyber operations (ACO) performance remains underexplored. Current agent-based security systems rely on either monolithic LLM prompts or rigid, static pipelines. We present *MaLO*, a configurable security platform to formalize and evaluate dynamic multi-agent graphs. MaLO provides four graph templates unified by a shared "prompt kernel" and tool interface: Single ReAct, Reasoner–Executor, Orchestrator–Workers, and Orchestrator-Workers with a Critic. We evaluate these architectures across a 42-task benchmark of varying difficulty quantified by our new SOCX metric, and a highly realistic, long-horizon security scenario (NSA CBC) that mirrors real-world task ambiguity. We provide the first systematic analysis of agent topology impact on security operations performance. Our results demonstrate that explicit task decomposition and orchestration significantly improve performance on medium and high-complexity security operations. By achieving a 78.6% success rate in our benchmark and solving tasks up to 40x faster than a human expert baseline, we show that architecture selection is a first-class design decision for autonomous cybersecurity agents.**

***Index Terms*—autonomous cyber operations, multi-agent systems, large language models, security benchmarking, agent topology**

I. Introduction

LLM-based agents have rapidly progressed from interactive assistants to systems that can plan, call tools, and execute multi-step workflows. In cybersecurity, this has sparked a wave of agentic frameworks for incident response training [1], penetration testing [2], vulnerability research [3], and solving Capture the Flag (CTF) challenges [4].

Most prior evaluations, however, implicitly conflate three factors: (i) the *model* (capability and tool-use fidelity), (ii) the *benchmark* (often self-contained CTF tasks with deterministic objectives), and (iii) the *agent topology* (single-agent, planner–executor, or fixed multi-agent hierarchies). This makes it difficult to attribute improvements to architectural choices rather than benchmark idiosyncrasies or model upgrades. Moreover, common CTF-style benchmarks underrepresent properties that dominate real security work: ambiguous problem statements, partial observability, competing hypotheses, and frequent false starts that require backtracking and re-planning.

This paper proposes an *evaluation infrastructure* that treats the agent topology as a first-class variable and tests agents on tasks that resemble real operations. We therefore introduce **MaLO**, a platform designed to (1) systematically evaluate different agent graphs under uniform interfaces and constraints, and (2) push evaluation beyond toy problems by including a realistic, long-horizon case study drawn from the NSA Codebreaker Challenge (CBC) [5].

Our contributions are as follows:

1) **Dynamic Multi-Agent Framework:** We introduce MaLO, a framework that performs security operations autonomously by allowing collaborative, custom agent topologies.
2) **Operation Complexity Standard:** We introduce the Security Operation Complexity Index (*SOCX*) to standardize the difficulty of interactive security tasks.
3) **Comprehensive Benchmarking:** We curate a 42-task benchmark and integrate the NSA CBC 2025 multi-stage investigation scenario to evaluate MaLO across both controlled, new and highly realistic security environments.
4) **Topology Impact Analysis:** We conduct the first comparative study on agent topology impact on security performance. We design four representative multi-agent graphs with standardized tools and kernel prompts for controlled comparisons, and evaluate them across tasks of varying complexity against a highly proficient human security expert.

We release all our artifacts as open source projects.[1,2]

II. Related Work

Recent studies explored the use of LLMs for security challenges, ranging from penetration testing, vulnerability research, to CTF challenges (see Table I). One of the earlier research efforts, PentestGPT [2], uses LLMs for penetration testing tasks; however, it heavily relies on human interaction for prompting and action execution. Then EnIGMA [6] proposed an automated interactive single agent focused on CTF challenges of different categories while providing an Agent-Computer Interface for execution. Similarly, AutoAttacker [7] simulates post-breach attacks by calling a single LLM with four separate prompt templates, a summarizer, planner, navigator, and a RAG experience manager. As shown, none is multi-agent: PentestGPT requires human-in-the-loop (HITL) between modules, EnIGMA is single-agent with assisted tooling, and AutoAttacker switches prompts in a single loop without inter-agent communication or independent LLM conversations with feedback.

[1] https://github.com/r4wd3r/malo
[2] https://github.com/r4wd3r/malo-benchmark

TABLE I: Comparative Analysis of Autonomous Security Frameworks. CC = Complexity Classification

Framework (Year)	Multi?	Arch.	CC	Goal
PentestGPT (2023) [2]	✗	Fixed	✗	CTF
AutoAttacker (2024) [7]	✗	Fixed	✗	Post-Exploitation
EnIGMA (2024) [6]	✗	Fixed	✓	CTF
D-CIPHER (2025) [8]	✓	Fixed	✓	CTF
CRAKEN (2025) [9]	✓	Fixed	✗	CTF
CVE Genie (2025) [10]	✓	Fixed	✗	CVE Reproduction
VulnBot (2025) [11]	✓	Fixed	✗	Subtask Completion
MaLO (2026)	✓	Dynamic	✓	General SecOps

As operational complexity grows, single-agent reasoning becomes brittle under long-horizon dependencies and ambiguous evidence. Multi-agent systems provide modular reasoning, parallel execution, and structured oversight that better align with real-world demands. A system is considered multi-agent if it has (1) multiple agents with distinct roles and responsibilities, (2) each agent is a separate LLM call maintaining its own conversation history or context, (3) interactions between agents exist beyond independent LLM calls, and (4) there is a feedback loop between agents [8].

Recent work employs multi-agent decompositions (e.g., planner–executor or role-separated agents) for a variety of security operations. D-CIPHER [8] creates a Planner-Executor system with multiple Executors. An Auto-prompt agent first explores the agent environment to create a better initial prompt for the Planner. D-CIPHER achieves state-of-the-art results on NYU CTF Bench (22%), Cybench (22.5%), and HackTheBox (44%). CRAKEN is an extension of D-CIPHER, with RAG-based improvements on the Planner-Executor interactions.

Multi-agent systems have also been proposed for vulnerability evaluation. CVE Genie [10] uses multi-agent collaboration in reproducing existing CVEs. On the other hand, VulnBot [11] breaks penetration testing down into three phases: reconnaissance, scanning, and exploitation. These phases are handled separately by agents that communicate through a Summarizer module and task dependencies managed by a Directed Acyclic Graph (DAG).

All previous frameworks demonstrate the capabilities of multi-agent architectures, yet they use fixed pipelines. We find that the literature offers limited evidence on *which topology* is preferable as operational complexity increases.

Another factor often dismissed in previous research is the complexity and real-world fidelity of the tasks the agents attempt. They generally treat tasks as independent items and rarely stratify results by operational complexity dimensions such as uncertainty and multi-stage depth.

MaLO complements prior work by focusing on *evaluation infrastructure* rather than a single, optimized architecture: it provides a common substrate to compare multiple topologies, introduces a complexity metric (SOCX) to stratify tasks beyond category labels, and evaluates on both a controlled benchmark and a realistic multi-stage setting (the NSA Codebreaker Competition) [5].

In the next section, we introduce MaLO's architecture and explain how it addresses some of the current challenges for autonomous security operations.

III. MaLO Architecture

MaLO is a Python multi-agent framework that autonomously performs security operations in the form of challenges (e.g., binary exploitation, reverse engineering, forensics). Each run loads (i) an operation description and artifacts (e.g., prompt, files, optional challenge container), (ii) a user-selected multi-agent graph with per-agent prompts and model settings, and (iii) execution limits (timeouts, turn caps). The system then iterates between LLM reasoning and tool calls until success or termination.

We designed MaLO under seven *Design Principles (DPs)* summarized in Table II: dynamic topology, batched and parallel execution, safe action handling, user-defined halting, per-agent configuration, context isolation, and observability.

TABLE II: MaLO Design Principles (DPs).

ID	Principle	Requirement
DP1	Dynamic topology	Configurable multi-agent graphs for reasoning and execution.
DP2	Operation batching	Single, multiple, and parallel runs across environments.
DP3	Action execution	Safe and correct interaction with the testing environment.
DP4	User-defined halting	Stop on user constraints (turns, tokens, timeouts) with cataloged reasons.
DP5	Per-agent configuration	Granular prompts, hyperparameters, models, and providers per agent.
DP6	Context isolation	Hide goal from reasoning context; truncate message growth.
DP7	Observability	Log all operations; collect traces and trajectories.

Fig. 1 describes the general architecture of MaLO. It receives the initial settings from the user through Command-Line Interface (CLI) arguments and configuration files. It ❶ loads the challenge data (e.g., prompt, files, goal, category, optional container file), the dynamic multi-agent graph configuration ❷ with per-agent settings, the limits configurations, and the *Testing Environment* container. Then ❸ deploys the *Testing Environment* for safe autonomous code execution and ❹ the desired multi-agent graph with the available tools. Then ❺ it starts the challenge run with the defined graph and termination constraints. MaLO iterates by allowing the agents in the graph to reason with their specified LLM and execute tool calls that interact with the *Testing Environment*. The run terminates when the defined goal is successfully achieved, the multi-agent graph gives up, or one of the timeout or max turns constraints is met.

Table III summarizes the seven MaLO modules and the design principles each addresses.

Now we will explain how MaLO dynamic graphs work and how they interact with the different tools and the environment.

A. Custom Graphs for Dynamic Topology

To define a configurable and dynamic multi-agent reasoning system, we explored current multi-agent frameworks. The most popular available include LangGraph [15], Microsoft Autogen (renamed to Microsoft Agent Framework) [17], and CrewAI

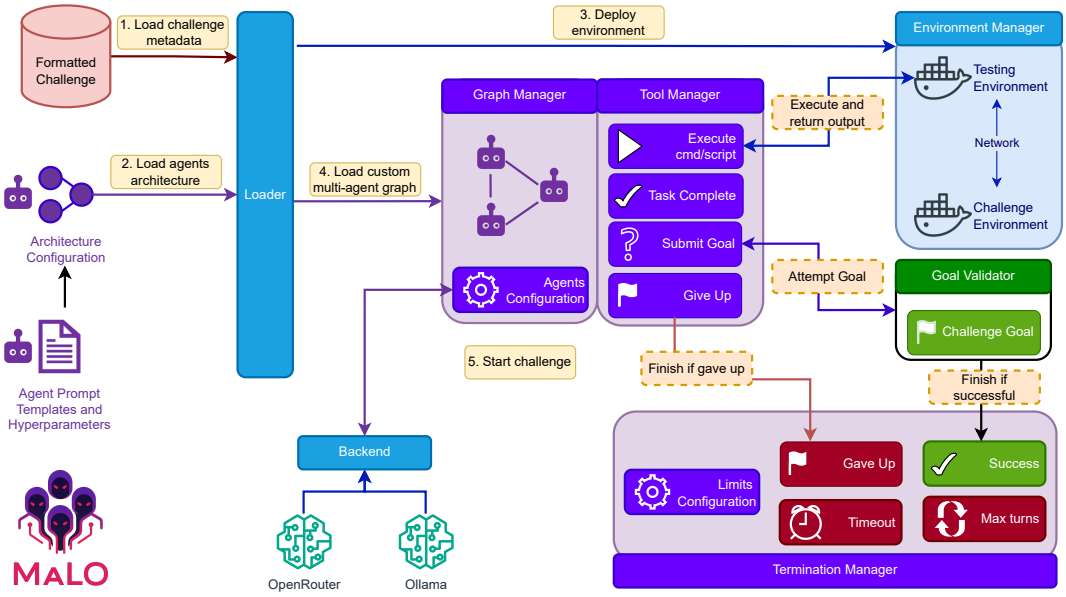


Fig. 1: MaLO Architecture.

TABLE III: MaLO modules and design principles they address.

Module	DPs	Role
Loader	DP1–DP6	Loads settings, multi-agent graph (YAML), execution constraints, and challenge data.
Backend	DP5, DP7	Ollama [12] and OpenRouter [13]; W&B [14] trace and trajectory logging.
Graph Manager	DP1–DP5	Workflows (ReAct, Orchestrator-Workers, etc.) on LangGraph [15].
Tool Manager	DP2, DP3	Async tool execution under OpenAI JSON schemas [16]; `execute_script`, `submit_goal`, `give_up`.
Env. Manager	DP3, DP6	Docker container lifecycle (Kali Linux); non-interactive, isolated execution.
Goal Validator	DP4, DP6	Exact, list, and regex match validation, isolated from the execution environment.
Term. Manager	DP2, DP4, DP6, DP7	Stop conditions (timeouts, max turns, success) and final termination reasons.

[18]. We use LangGraph since it allows us to deploy finite-state custom graphs with high granularity, despite its node-edge definition complexity. Moreover, it facilitates the deployment of parallel workers through its API using the Map-Reduce pattern [15].

Prompting: For fairness, we want agents' prompts across graphs to be comparable. We share among agents a *minimal prompt kernel* with similar wording that varies slightly per workflow. They all share a *Role* [19] that is static for most agents (e.g., security researcher, planner), and dynamically assigned to workers in orchestrator workflows; an *Objective*, a placeholder replaced with each challenge prompt, context, and expected goal; *Environment description* describing its interaction with other agents or the *Testing Environment* (e.g., running with high privileges on container and non-interactive commands); and *Tools* available for the agent with a brief description. For correctness, we also added *Guidelines* for each agent that describe the expected workflow in natural language.

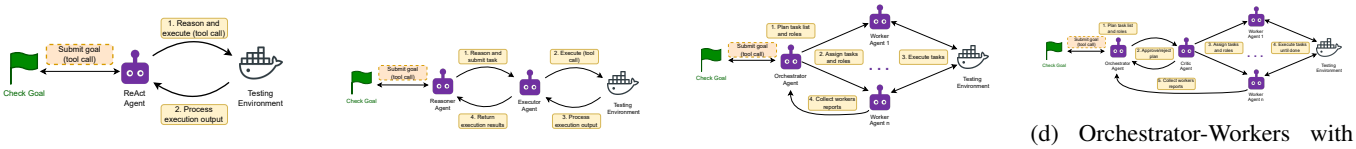
Interfacing: To ensure proper transition between nodes and tool calls, all graphs rely on structured JSON for model output and validation against Pydantic schemas and OpenAI-style JSON tool schemas [16]. For instance, for orchestrator graphs, outputs are validated to obtain state updates and task lists, and Critic responses to enforce `APPROVE`/`REJECT` semantics.

Dynamic Graphs: Fig. 2 shows our implemented multi-agent graphs exemplifying calls to `submit_goal` and `execute_script` after they receive the data of the challenge and the environment is deployed.

Single React Graph (SR) (Fig. 2a): Based on Reasoning and Act (ReAct) [20], a single agent alternates between reasoning and operation. As seen in Fig. 2a, ❶ the ReAct agent reasons and performs a tool call to interact with the environment, and then ❷ it receives the output to continue its execution; when it considers that it found a possible goal, it calls `submit_goal`. This recreates current ReAct coding agents (e.g., Codex [21], Claude Code [22], Gemini CLI [23]) in a comparable context with other multi-agent architectures.

Reasoner Executor Graph (RE) (Fig. 2b): Delegate the reasoning to one agent, which ❶ emits structured commands to an executor one. Then, ❷ executor runs them, ❸ analyzes the output, and ❹ feeds back the results to the reasoner until termination (i.e., termination constraints met, such as the reasoner `submit_goal` call successful). Our goal is to evaluate the benefit of splitting a ReAct single-agent approach into two and evaluate its performance.

Orchestrator-Workers Graph (OW) (Fig. 2c): Inspired by the Trail of Bits Buttercup Cyber-Reasoning System, an efficient architecture for vulnerability discovery and patching shown at DARPA's AIxCC competition [3], we propose an orchestrator-



(a) Single ReAct (SR) (b) Reasoner Executor (RE) (c) Orchestrator-Workers (OW) (d) Orchestrator-Workers with Critic (OC)

Fig. 2: Comparative analysis of different MaLO graph architectures.

workers architecture. ❶ An orchestrator plans and decomposes tasks, and ❷ dispatches them to parallel workers and assigns them roles. ❸ Each worker iterates until finishing the task (i.e., call `task_complete` or reaching their maximum subturns), and aggregates their report before iterating. Finally, ❹ the orchestrator collects worker reports to update its state and plan the next tasks.

Orchestrator architectures use reducer annotations to merge concurrent updates safely. Worker reports are merged via a list reducer that appends reports. We define a current state for the orchestrator for agent's self awareness, including a phase (reconnaissance, analysis, post-analysis, and complete), relevant findings from workers reports, a current hypothesis to describe what it believes is next to test, and a list of planned next tasks.

The orchestrator emits two structured records: a *task list* (each task carries `type`, `description`, `details`, `role`, and an optional `answer` for `submit` tasks) and a *state update* carrying `phase` (`reconnaissance`, `analysis`, `post-analysis`, `complete`), `findings`, `attempted`, `current_hypothesis`, and `next_tasks`.

Orchestrator-Workers with Critic Graph (OC) (Fig. 2d): Following recent advances in LLM reasoning [24] and Kahneman's System 1 / System 2 approach [25], we add a Critic that evaluates the orchestrator's plan and emits a `verdict` (`APPROVE` or `REJECT`) with optional `feedback` and `suggestion` fields. ❷ Workers execute approved tasks before looping with aggregated findings. To prevent Critic loops, after three consecutive rejections of the orchestrator's plan we force approval.

We discussed how to include dynamic multi-agent architectures in MaLO. Now we discuss how we validate goals and termination conditions.

B. Goal Validation and Termination Conditions

We must ensure that each challenge run has predefined termination reasons. We then define fatal and non-fatal events that are triggered by the *Termination Manager* as seen in Table IV.

Fatal events force the run termination and involve timeouts, successful goal submissions, or maximum turns reached. Non-fatal events involve subgraph timeouts or worker task completions. For these events, we do not propagate terminations, since their output can be used by adjacent agents to continue the execution process. For instance, a worker timeout in an orchestrator graph might indicate a non-interactive or non-responsive task, steering the planning toward different goals.

TABLE IV: Fatal vs non-fatal events. *SR: single_react, RE: reasoner_executor, OW: orchestrator_workers, OCW: orchestrator_critic_workers.*

Fatal Events	**Meaning**	**Where**
`success`	Goal validated by `submit_goal`	All
`give_up`	Agent explicitly stopped	All
`turn_limit`	Max turns reached	All
`terminated`	Tool or internal termination	All
`no_messages`	No messages to process	SR
`tool_timeout`	Tool execution timed out	SR
`challenge_timeout`	Overall run timeout exceeded	All
`missing_goal`	Goal validator missing	All
`error`	Exception or runtime error	All
Non-fatal Events	**Meaning**	**Where**
`executor_timeout`	LLM timed out	RE
`critic_timeout`	Treated as APPROVE	OC
`worker_timeout`	Worker report marked timed out	OW/OC
`worker_turn_limit`	Max worker turns reached	OW/OC
`tool_timeout`	Tool execution timed out	Worker subgraph
`task_complete`	Ends worker task with report	Worker tool

For turn limits, we consider the fatal event `turn_limit` only for main turns in multi-agent graphs; this is, reasoning turns (i.e., orchestrator in orchestrator workflows, and reasoner in reasoner-executor workflows).

To validate the goal correctness, MaLO allows three types of goal definition for the challenge loaded: *exact match, list match*, and *regex match*. As seen in Section IV, we ensure a proper goal definition in each of the challenges loaded by loading it isolated within the *Goal Validator* and interface with the tool call `submit_goal`.

Having a clearly defined implementation for the termination and management of fatal/non-fatal events allows us to better analyze each operation run with MaLO. Now we discuss the evaluation of our framework.

IV. Metrics and Benchmarks

In this section, we discuss how we classify the security operation complexity with our new *SOCX* standard, and the benchmarks we used to evaluate MaLO.

A. Security Operation Complexity Index

Our evaluation target is an *end-to-end security operation* that resembles real-world, where the operator (human or agent) interprets incomplete evidence, forms and revises hypotheses, and chains steps across interacting systems. Difficulty is dominated by *operational search*: ambiguous goals, plausible decoys, partial observability, and false starts that demand backtracking.

CTF difficulty labels, typically inferred from previous solvers' feedback, do not generalize to operations where attempts are sparse and solver populations change. The fail

to explain why a task is hard, which may involve multi-stage chaining, ambiguity, domain expertise, and prior exposure.

To support principled benchmarking of autonomous security systems on realistic tasks, we propose the *Security Operation Complexity Index (SOCX)*, a rubric-driven metric that decomposes operational complexity into separable dimensions. *SOCX* explicitly models sources of complexity that are central to real-world operations but are underrepresented in traditional CTF scoring. We synthesize *SOCX* from established rubrics, including vulnerability scoring (CVSS [26]), adversary taxonomies (MITRE ATT&CK [27]), cybersecurity workforce competencies (NIST NICE [28]), cognitive-load hierarchies (Bloom's taxonomy [29]), and our expertise in security operations.

Our proposed *SOCX* dimensions are defined as follows:

Environmental Complexity (EC): Describes the complexity of the execution context, including interactions and infrastructure (analogous to CVSS attack complexity).

Operational Depth (OD): Describes the amount and type of sequential steps/phases required to complete the task (analogous to multi-step chaining in MITRE ATT&CK).

Operational Uncertainty (OU): Degree of ambiguity and deception in the operation, including under-specified goals, misleading artifacts/decoys, partial observability, and the expected need for hypothesis-testing, backtracking, and recovery from false starts (analogous to Bloom's taxonomy [29], hypothesis revision before procedural recall).

Domain Complexity (DC): Describes the domain-specific knowledge or skill required to operate (analogous to specialty-area knowledge described in NIST NICE [28]).

Completion Rate (CR): Describes the solving or completion frequency of each operation. If the operations have not been performed before, this score is high (analogous to scoring conventions in CTF competitions [30]).

The Security Operation Complexity Index (S_{socx}) is the weighted arithmetic mean of the five dimensions of complexity, scaled to a percentage:

$$S_{socx} = \frac{100}{3} \sum_{i=1}^{5} w_i \cdot d_i \tag{1}$$

where w_i represents the weight of the i-th dimension such that $\sum w_i = 1$ and d_i represents the score of the i-th dimension (EC, OD, OU, DC, CR) $\in \{0, 1, 2, 3\}$.

SOCX values range from 0 to 100. We consider the complexity **low** if *SOCX* is less than 40, **medium** if it is greater or equal to 40 and less than 70, **high** if it is greater or equal to 70 and less than 85, and **extreme** if it is greater or equal to 85.

As in CVSS [26], we adjusted the weight of each dimension based on iterative tuning. To propose relevant weights and scores, we relied on two experts to categorize the complexity of the challenges, including the one who solved them correctly. To mitigate expert biases, we created an LLM module as a "third expert" with Retrieval-Augmented Generation (RAG) [31] to query the latest OpenAI model `gpt-5.2` with a description of the *SOCX* classification and its dimensions, and provided each challenge's prompt, write-up, and metadata for the module to provide values for each dimension. We tuned the weights for each dimension by averaging the scores.

B. MaLO Security Benchmark

Measuring the security capabilities of foundational models accurately is an open challenge. CTF challenges serve as critical validation of the agent-based framework's capabilities in security operations. Multiple benchmarks [4], [32], [33] have been published with CTF challenges and security operations, yet they do not provide a systematic classification of their complexity and, more importantly, include well-known challenges with public writeups that are then ingested by LLMs, leading to data contamination between training and testing. We create our own MaLO Benchmark with **42 challenges**, categorized their complexity via *SOCX* **(16 as low, 22 as medium, 4 as high)**, and include new tailored challenges to prevent training data contamination with known problems.

The challenges were explicitly developed and gathered to simulate real-world security patterns in operational settings, making them particularly relevant for assessing practical utility within real target systems. We built it oriented to exact flag challenges, following a structure based on the one proposed by CTFCli [30] for scalable and structure challenge storage. Each challenge contains its description, resources (e.g., files and URLs), category, user-defined tags for customized classification, and solution data (i.e., hints, flags, step-by step writeup). They vary in complexity, goals, and objectives, and include problems from competitions such as PicoCTF [34] and CSAW [35].

Success on these challenges demonstrates our framework's ability to apply standard security strategies to previously unknown problems. These initial test cases help establish a baseline for our ongoing development while providing concrete examples of our framework's current capabilities in solving realistic security problems.

C. NSA Codebreaker Challenge 2025

The 2025 NSA Codebreaker Challenge (CBC) [5] is framed as a blue-team investigation that evolves into an offensive operation against a sophisticated, multi-stage intrusion targeting a fictional defense-related network. Over the course of seven tasks, participants move from analyzing a single suspicious artifact on a development host to exploiting the adversary's own infrastructure.

The central problem is to reconstruct the adversary's operational story while simultaneously building the capability to counterattack. Each task exposes one layer of this campaign while building toward offensive capability; taken together, they form a narrative that transitions from incident response to offense, reflecting the dual nature of modern security operations against advanced persistent threats.

From an evaluation standpoint, CBC is particularly valuable because it embodies the operational characteristics that SOCX is designed to capture: environmental coupling (e.g., interaction

TABLE V: NSA CBC Challenges: Problems, solutions and complexity factors

Task	Description	Solution	Challenges
1	**Forensics:** Identify suspicious artifacts on a compromised Linux machine image.	Analyze `.bash_history` to reconstruct the attacker's command sequence; identify a residual config file.	Command history correlation with filesystem state. Distinguish legitimate files from malware through shell context analysis.
2	**Network Analysis:** Find compromised devices from PCAP with anomalous traffic.	Correlate DNS queries, FTP backups, and custom TCP handshakes. Dump configuration from FTP traffic.	Custom protocol connection was a decoy. Requires multiple protocol stream analysis (DNS, FTP, TCP).
3	**Memory Forensics:** Find the attacker's pivot binary from a memory dump.	Recover deleted binary from memory pages. Decrypt XOR-obfuscated strings to find DNS poisoning configuration	Requires kernel symbol handling. Fileless malware prevents standard disk analysis.
4	**Malware RE:** Deobfuscate binary with anti-debugging techniques to find a file path.	Bypass `ptrace` checks with `LD_PRELOAD` hooking. Extract the second-stage payload via dynamic analysis. Decrypt RC4 strings.	Dynamic instrumentation/hooking for anti-debugging. Multi-stage loading `memfd_create` interferes with static analysis.
5	**Cryptography:** Decrypt C2 traffic secured with custom double-AES protocol.	RE and identify a key generation flaw (keys masked to 26 bits). Perform Meet-in-the-Middle attack to recover keys.	Recognize implementation flaws (low entropy) over theory. Brute force (2^{52}) is infeasible.
6	**Access Control:** Privilege escalation on chat application and access a private channel.	Exploit logic flaw in "negotiation" bot. Chain commands to manipulate user groups.	Multi-hop approach with intermediate steps. Requires chain actions planning and bot emulation rather than a single exploit primitive.
7	**Mobile Exploit:** Create malicious payload to compromise an Android app in 1 attempt.	Discover and exploit a path traversal vulnerability with dynamic code loading to overwrite a trusted plugin JAR with malicious DEX code.	Requires understanding Android (Dalvik/DEX) architecture. Complex payload construction to trigger logic flaw.

TABLE VI: Example SOCX Ratings for NSA CBC Tasks

Task	EC	OD	OU	DC	CR	Score	Complexity
Task 1	2	2	1	1	2	50.5	Medium
Task 2	2	3	2	1	2	69.4	Medium
Task 3	2	3	2	2	3	77.4	High
Task 4	2	3	2	2	3	77.4	High
Task 5	2	3	2	3	3	82.8	High
Task 6	2	3	3	2	3	88.2	Extreme
Task 7	2	3	3	3	3	93.5	Extreme
Weights	0.194	0.242	0.323	0.161	0.080		

between hosts, memory images, and network traces), multi-phase depth, adversarial deception, and the need for iterative refinement under partial observability.

Compatibility: At the time of this research, the NSA CBC challenge was open, and no solutions were publicly available (so LLMs had not been trained with the solutions to these problems). Astra Tsai was the first student to solve all the challenges, and she documented her walkthrough and timed her performance for each task. With these resources, we adapted 6 out of the 7 NSA CBC challenges for MaLO, including the additional context from prior tasks needed for the final challenges. Challenge 7 was not adapted due to its custom exploit evaluator.

Table V shows an overview of the NSA CBC 2025 challenges, and what makes them complex and realistic. Table VI shows the *SOCX* classification for the NSA CBC 2025 challenges and the adjusted weights.

V. Evaluation

Table VII summarizes the experimental setup. We use a single model across all agents to isolate the effect of topology from model capability, and pick limits consistent with prior agentic security frameworks [4], [6], [8]–[10]. All agents share the minimal prompt kernel from Section III-A with JSON schema validation enforced on graph state updates and tool calls. When a challenge ships an auxiliary container, we deploy it on the same Docker network as the testing environment and pass its hostname through the prompt kernel's *{objective}* placeholder.

TABLE VII: Experimental setup.

Component	Parameter	Value
Model	LLM (all agents)	`gemini-3-flash-preview` via OpenRouter [13]
	Temperature	0.2 [6]
Limits	`max_turns` / `worker_max_turns`	50 / 10
	Parallel workers (orchestrator)	3
	`challenge_timeout` / `worker_timeout`	1800 s / 300 s
	`recursion_limit`	2000
	`observation_truncation_chars`	5000
Env.	Host OS	Ubuntu Server 22.04 x64 with Docker [36]
	Testing container	Kali Linux x64 (root); web, binary, exploit, and forensics tools
Obs.	Tracing	W&B/Weave [14] per-call traces and trajectories; termination reasons, turn counts, tokens, and cost are logged per challenge

A. Benchmarking

We evaluate MaLO against our malo-benchmark with **42 challenges (SOCX Complexity: 4 high, 22 medium, and 16 low)**. Table VIII shows the success rates per architecture. The orchestrator worker graph (OW) solved the most challenges (33/42 - 78.6%), which suggests that task decomposition and orchestration is effective for security operations. Moreover, it

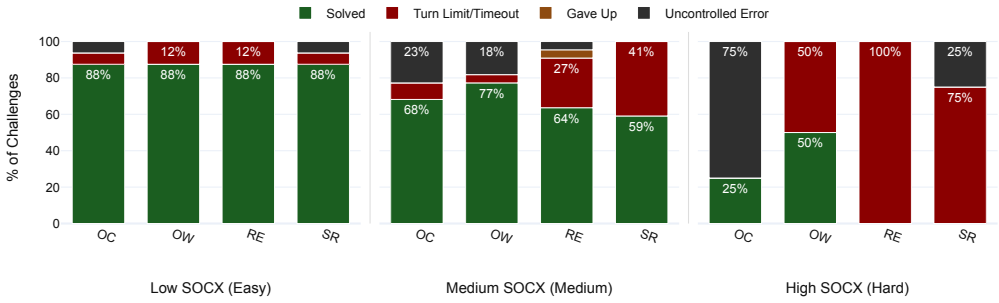


Fig. 3: Benchmark with malo-benchmark challenges, classified by architecture and SOCX complexity.

was the only one able to solve 2 out of 4 highly-complex challenges, which advocates for its usage on complex security operations. The orchestrator critic graph (OC) followed second (30/42 - 71.4%), which was intended to improve planning but resulted in a lower success rate. Single React (SR) had the lowest success rate (27/42 - 64.3%), which indicates that a single agent managing reasoning and execution might be less robust in security operations.

TABLE VIII: Summary of success metrics across MaLO graphs vs malo-benchmark. *SR = Single ReAct, RE = Reasoner Executor, OW = Orchestrator Workers, OC = Orchestrator Workers with Critic, RT = Reasoning Turns*

Metric	SR	RE	OW	OC
Solve Rate (Total)	64.3%	66.7%	**78.6%**	71.4%
Avg. Tokens / Solve (RT)	102150.11	57541.04	**26079.79**	27322.27
Median Tokens / Solve (RT)	27218.00	12400.00	9819.00	**3389.50**
Avg. Time to Solve (s)	**39.00**	55.97	138.48	133.63
Median Time to Solve (s)	17.86	**17.23**	39.37	27.00
Success @ High SOCX	0.0%	0.0%	**50.0%**	25.0%

In terms of time efficiency, SR demonstrated the least average solving time for the tasks it was capable of solving. Although we do not consider the time on unsolved challenges, this might indicate that SR can be leveraged for certain tasks. This is relevant, considering that the average solving time for the successful OC graph was the highest (approximately 243% more than SR).

Fig. 3 groups the malo-benchmark challenges by SOCX class and shows the termination reason for each (success, turn or timeout limits, give-up, and uncontrolled errors traced to container interaction issues, JSON decoding under long context, or undiagnosed faults). Under the defined constraints, RE was the only architecture that ever invoked `give_up`. **Insight:** all architectures solved the same low-complexity tasks, so for easy operations Single ReAct is the best choice on time efficiency (Table VIII); separating reasoning from execution improves success across graphs, and orchestrator graphs (OW, OC) extend coverage to hard tasks at the cost of longer runtimes.

To perform a fair comparison between reasoning token consumption, we consider in orchestrator graphs the token usage for reasoning turns. In other words, to preserve a token usage evaluation in reasoning as fair across architectures, we do not add the workers' token consumption since they work as subgraphs that increase this metric dramatically.

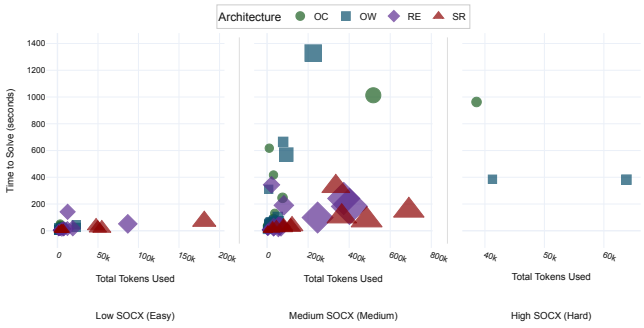


Fig. 4: Token consumption in reasoning turns vs time to solve per architecture

Fig. 4 plots token consumption for reasoning turns against time to solve per architecture. SR shows an edge case where one challenge consumes the maximum registered tokens in 200 seconds, suggesting that ReAct's tendency to retain long outputs increases token use even with character truncation in place. OC's iterative plan evaluation following a Critic rejection produced the second-highest consumption. **Insight:** standard schemas reduce reasoning-turn tokens without hurting success, since SR's relaxed prompts access full context but do not solve additional tasks, and output abstractions reduce tokens with some risk of hallucination and context loss during summarization.

Based on our results, orchestrator graphs are better suited for medium to high complexity security operations. In particular, when using non-frontier models such as `gemini-3-flash-preview`, our results show that orchestrator graphs have a higher success rate despite a longer runtime execution when

performing security operations. We then use the most robust multi-agent architecture (orchestrator-worker), to try to solve the NSA CodeBreaker Challenges 2025 and compare it to the security expert who solved it in record time.

B. MaLO vs Human Expert

We evaluate MaLO against 6 of the 7 NSA CBC 2025 challenges. As we mentioned in Section IV-C, task 7 requires a customized evaluator that accepts any exploit that triggers the specific vulnerability, and MaLO currently does not support this goal validation approach. We compare MaLO with the security expert who first solved these challenges, who we asked to time and document each challenge solving process. Table IX shows the timed success rate for human (security expert), MaLO with Orchestrator-Worker (OW), and MaLO OW with hints. We ensure the hints do not provide the correct flag, but guide towards the path to find it.

TABLE IX: Performance Comparison - Human Expert vs. MaLO. ✓ = Success, ✗ 🕒 = Timeout, ✗ ✋ = Turn Limit.

Challenge (SOCX)	Human	MaLO OW	MaLO OW (Hints)
CBC 1 (Medium)	1h 42m 24s ✓	2m 33s ✓	46s ✓
CBC 2 (Medium)	4h 25m 06s ✓	10m 57s ✗ ✋	3m 5s ✓
CBC 3 (High)	4h 27m 46s ✓	27m 38s ✗ 🕒	13m 43s ✓
CBC 4 (High)	4h 37m 33s ✓	18m 55s ✗ 🕒	19m 44s ✗ 🕒
CBC 5 (High)	2h 26m 02s ✓	21m 10s ✗ 🕒	17m 24s ✗ ✋
CBC 6 (Extreme)	3h 59m 04s ✓	10m 23s ✗ ✋	26m 9s ✗ 🕒
CBC 7 (Extreme)	13h 09m 46s ✓	N/A	N/A

MaLO was able to solve the first task **40.16x** times faster than the expert without any hints. Nevertheless, MaLO did not solve the subsequent tasks by itself. When guided to the real answer, MaLO solved tasks 2 and 3 in between **19.52x and 85.98x** times faster than the human, but was not able to solve the remaining high complexity tasks.

Unsolved Challenges: We analyzed the failed trajectories. *(1) Context pollution*: in Task 5, MaLO tried to recover the goal from residual hallucinated context from Task 2; in Task 6 it found relevant user accounts but assumed commands needed obviously malicious strings due to interference with prior conversations. *(2) Tooling limitation*: MaLO detected anti-debugging in Task 4 but timed out trying to bypass it without non-interactive specialized tools. *(3) Strategic reasoning*: in Task 2 it bruteforced the goal until the turn limit instead of changing strategies; in Task 3 it could not deduce that fileless malware had been used to avoid disk artifacts. **Insight:** current LLM-based agents do not yet match expert strategic reasoning on complex security operations; they are bound by interactive-only tooling (e.g., GUI disassemblers and decompilers) and degrade under context drift, with context sanitization (refreshing agent state) and persistent long-term memory (externalized findings summaries) as direct mitigations.

VI. Conclusions

In this paper, we introduced MaLO, demonstrating how LLM-based agents can collaborate to autonomously perform complex security operations. We release a new benchmark with 42 challenges, and systematically classified their complexity with our novel Security Operation Complexity Index (*SOCX*). We show that task decomposition with Orchestrator-Workers can autonomously solve 78.6% of our benchmark, with non-frontier LLMs. Finally, we evaluate MaLO against a security expert using the NSA CBC 2025 challenge and show that these systems can surpass skilled human experts on low-medium complexity tasks (40x faster solving), but lack the strategic reasoning for highly challenging operations. As limitations and future work ideas, we discuss:

Increasing experimental scenarios: We can use our framework to document more experiments with different foundational models and state-of-the-art benchmarks [4], [33]. We can also compare MaLO with other frameworks to evaluate its performance with these benchmarks.

Improving tool support: We can improve the testing environment and allow the framework to execute more elaborate actions. We can improve goal validation for complex cases, including exploitation emulators and LLM as judge agents for non-CTF operations.

Persistent memory: Summarization through persistent memory is effective for context management in planning and agents' state updates in coding agents [21], [22]. We can integrate this so agents document findings and milestones in external sources instead of their conversational context.

Our work suggests that the path to fully autonomous security is not dependent merely on larger foundational models, but also in sophisticated agent architectural orchestration. We believe platforms like MaLO and standards like *SOCX* will be essential in defining the "human-in-the-loop" boundary in autonomous security systems, as they evolve toward solving high-complexity tasks and may transition the human role from active operator to high-level strategist.

Ethical Disclosure

CTF challenges have been proven to be an effective way to represent security operations and evaluate the security automation capabilities of foundational models [4], [6], [37]. Nevertheless, the use of these solutions can also aid adversaries in improving their capabilities.

The outcome results from this research are prone to misuse [38]. Due to the research goal we proposed to pursue for this research, some of our outcomes can also be used by adversaries. We encourage the readers to be aware of the potential misuses that foundational models can have, including the ones included in our research, and propose strategies for mitigating misuse.

We assisted our artifact development with AI for code refactoring suggestions and documentation. All code changes were reviewed, tested, and validated by the authors. We did not use AI to generate experimental results or analyses.

Acknowledgements

This work is based upon the work supported by the National Center for Transportation Cybersecurity and Resiliency (TraCR) (a U.S. Department of Transportation National

University Transportation Center) headquartered at Clemson University, Clemson, South Carolina, USA (USDOT Grant #69A3552344812). Any opinions, findings, conclusions, and recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of TraCR, and the U.S. Government assumes no liability for the contents or use thereof. We also thank the United States Cyber Command (USCYBERCOM) for their feedback on this research and Capt. Steve Sam for his valuable input. We collaborated with OpenAI, using their funding to advance our research in AI-driven cybersecurity solutions. We thank Oliver Pearce and Sotirios Kougiouris from UCSC for their contributions to the MaLO benchmark.

REFERENCES

[1] S. R. Castro, R. Campbell, N. Lau, O. Villalobos, J. Duan, and A. A. Cardenas, "Large language models are autonomous cyber defenders," in *2025 IEEE Conference on Artificial Intelligence (CAI)*, 2025, pp. 1125–1132.

[2] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: An LLM-empowered Automatic Penetration Testing Tool," Jun. 2024.

[3] Trail of Bits, *Buttercup: AI Cyber Reasoning System*, 2025. [Online]. Available: https://www.trailofbits.com/buttercup/

[4] M. Shao, S. Jancheska, M. Udeshi, B. Dolan-Gavitt, H. Xi, K. Milner, B. Chen, M. Yin, S. Garg, P. Krishnamurthy, F. Khorrami, R. Karri, and M. Shafique, "NYU CTF Dataset: A Scalable Open-Source Benchmark Dataset for Evaluating LLMs in Offensive Security," Jun. 2024.

[5] National Security Agency. (2025) NSA Codebreaker Challenge. [Online]. Available: https://www.nsa.gov/Press-Room/Press-Releases-Statements/Press-Release-View/Article/4312875/nsas-12th-annual-codebreaker-challenge-is-underway-for-2025/

[6] T. Abramovich, M. Udeshi, M. Shao, K. Lieret, H. Xi, K. Milner, S. Jancheska, J. Yang, C. E. Jimenez, F. Khorrami, P. Krishnamurthy, B. Dolan-Gavitt, M. Shafique, K. Narasimhan, R. Karri, and O. Press, "EnIGMA: Enhanced Interactive Generative Model Agent for CTF Challenges," Sep. 2024.

[7] J. Xu, J. W. Stokes, G. McDonald, X. Bai, D. Marshall, S. Wang, A. Swaminathan, and Z. Li, "Autoattacker: A large language model guided system to implement automatic cyber-attacks," 2024. [Online]. Available: https://arxiv.org/abs/2403.01038

[8] M. Udeshi, M. Shao, H. Xi, N. Rani, K. Milner, V. S. C. Putrevu, B. Dolan-Gavitt, S. K. Shukla, P. Krishnamurthy, F. Khorrami, R. Karri, and M. Shafique, "D-cipher: Dynamic collaborative intelligent multi-agent system with planner and heterogeneous executors for offensive security," 2025. [Online]. Available: https://arxiv.org/abs/2502.10931

[9] M. Shao, H. Xi, N. Rani, M. Udeshi, V. S. C. Putrevu, K. Milner, B. Dolan-Gavitt, S. K. Shukla, P. Krishnamurthy, F. Khorrami, R. Karri, and M. Shafique, "Craken: Cybersecurity llm agent with knowledge-based execution," 2025. [Online]. Available: https://arxiv.org/abs/2505.17107

[10] S. Ullah, P. Balasubramanian, W. Guo, A. Burnett, H. Pearce, C. Kruegel, G. Vigna, and G. Stringhini, "From cve entries to verifiable exploits: An automated multi-agent framework for reproducing cves," 2025. [Online]. Available: https://arxiv.org/abs/2509.01835

[11] H. Kong, D. Hu, J. Ge, L. Li, T. Li, and B. Wu, "Vulnbot: Autonomous penetration testing for a multi-agent collaborative framework," 2025. [Online]. Available: https://arxiv.org/abs/2501.13411

[12] Ollama, *Ollama: Get Up and Running with Large Language Models*, 2026. [Online]. Available: https://ollama.com/

[13] OpenRouter, *OpenRouter Documentation*, 2026. [Online]. Available: https://openrouter.ai/docs

[14] Weights & Biases, *Weights & Biases: The AI Developer Platform*, 2026. [Online]. Available: https://wandb.ai/site/

[15] LangChain, "LangGraph Documentation," https://langchain-ai.github.io/langgraph/tutorials/introduction/, 2025.

[16] OpenAI, *Structured Outputs - OpenAI API*, 2026. [Online]. Available: https://platform.openai.com/docs/guides/structured-outputs

[17] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," 2023.

[18] CrewAI, "CrewAI Documentation," https://docs.crewai.com/introduction, 2025, accessed: 2025-02-04.

[19] A. Kong, S. Zhao, H. Chen, Q. Li, Y. Qin, R. Sun, X. Zhou, E. Wang, and X. Dong, "Better Zero-Shot Reasoning with Role-Play Prompting," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2024, pp. 4099–4113.

[20] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X

[21] OpenAI, *Codex CLI: AI Coding Agent for the Terminal*, 2026. [Online]. Available: https://github.com/openai/codex

[22] Anthropic, *Claude Code: Agentic Coding Tool*, 2026. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code/overview

[23] Google, *Gemini CLI: Google's Open Source AI Agent*, 2026. [Online]. Available: https://github.com/google-gemini/gemini-cli

[24] D. Zhang, Z.-Z. Li, M.-L. Zhang, J. Zhang, Z. Liu, Y. Yao, H. Xu, J. Zheng, X. Chen, Y. Zhang, F. Yin, J. Dong, Z. Guo, L. Song, and C.-L. Liu, " From System 1 to System 2: A Survey of Reasoning Large Language Models ," *IEEE Transactions on Pattern Analysis & Machine Intelligence*, no. 01, pp. 1–20, Nov. 2025. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/TPAMI.2025.3637037

[25] D. Kahneman, *Thinking, Fast and Slow*. New York: Farrar, Straus and Giroux, 2011.

[26] Forum of Incident Response and Security Teams (FIRST), *Common Vulnerability Scoring System v4.0 Specification Document*, 2023. [Online]. Available: https://www.first.org/cvss/v4.0/specification-document

[27] Mitre, *MITRE ATT&CK Framework*, Std. [Online]. Available: https://attack.mitre.org/

[28] R. Petersen, D. Santos, M. C. Smith, K. A. Wetzel, and G. Witte, "Workforce framework for cybersecurity (nice framework)," 2020.

[29] L. W. Anderson and D. R. Krathwohl, *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, 2001.

[30] CTFd, "Overview | CTFd Docs," https://docs.ctfd.io/docs/management/ctfcli/overview/, 2025.

[31] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS '20. Red Hook, NY, USA: Curran Associates Inc., 2020.

[32] XBOW. (2024) Xbow: Ai-powered penetration testing platform. Automated offensive security and vulnerability research. [Online]. Available: https://xbow.com/

[33] A. K. Zhang, N. Perry, R. Dulepet, J. Ji, J. W. Lin, E. Jones, C. Menders, G. Hussein, S. Liu, D. Jasper, P. Peetathawatchai, A. Glenn, V. Sivashankar, D. Zamoshchin, L. Glikbarg, D. Askaryar, M. Yang, T. Zhang, R. Alluri, N. Tran, R. Sangpisit, P. Yiorkadjis, K. Osele, G. Raghupathi, D. Boneh, D. E. Ho, and P. Liang, "Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models," Oct. 2024.

[34] P. Chapman, J. Burket, and D. Brumley, "PicoCTF: A Game-Based Computer Security Competition for High School Students," in *2014 USENIX Summit on Gaming, Games, and Gamification in Security Education (3GSE 14)*, 2014.

[35] K. Chung and J. Cohen, "Learning Obstacles in the Capture The Flag Model," in *2014 USENIX Summit on Gaming, Games, and Gamification in Security Education (3GSE 14)*, 2014.

[36] Docker Inc., *Docker: Accelerated Container Application Development*, 2026. [Online]. Available: https://www.docker.com/

[37] R. Fang, R. Bindu, A. Gupta, Q. Zhan, and D. Kang, "Teams of LLM Agents can Exploit Zero-Day Vulnerabilities," Jun. 2024.

[38] S. L. Schröer, G. Apruzzese, S. Human, P. Laskov, H. S. Anderson, E. W. N. Bernroider *et al.*, "SoK: On the Offensive Potential of AI," 2024.